



Lab Sheet 01 - Identifying and Reasoning About Quality Attributes

Objective

The objective of this practical is to help students recognize quality attributes as the primary drivers of architectural decisions. By the end of this activity, you should be able to identify architecturally significant quality attributes for a given system, express them as clear and testable scenarios, and reason for trade-offs between competing attributes.

- This is a self-guided activity.
- You may attempt this as a pair. Discussions are encouraged.
- You may consult the lab instructors at any time, but you are expected to carry out the steps on your own (as a pair).
- You are free to refer to any material online/offline.

Before You Begin

Most architectural decisions are driven not only by functionality but by quality attributes (architecture characteristics) such as availability, performance, security, and modifiability that determine **how well a system behaves under real operating conditions**. Functional requirements state **what a system should do** (e.g., "patients can book appointments"); quality attributes state **how well it should do those** (e.g., "... while remaining responsive during peak booking periods — for instance, responding to a GET request within 500ms under peak load conditions such as 1,000 concurrent users"). These matter to architects because they are difficult to add later and often trade off against one another.

As you read the case study, focus on identifying which statements describe conditions or expectations that would genuinely shape the architecture, rather than details that are purely functional or incidental. Not every sentence carries architectural significance.

Task

Case Study: SmartClinic – Multi-Branch Patient Appointment and Records Platform

A healthcare technology company is building a platform to manage patient appointments and medical records for a network of clinics operating across several cities. Patients should be able to book appointments, view past visit history, and receive reminders. Clinic staff need to manage schedules, update patient records during and after consultations, and generate visit summaries. Staff have varying levels of familiarity with digital systems — some have used similar software before, while others are using a system like this for the first time — so they need to be able to learn the platform quickly and correct mistakes, such as an accidental cancellation or an incorrect entry, without complicated recovery steps.

The platform stores sensitive medical information and must comply with healthcare data protection regulations. Usage is uneven throughout the day, with sharp spikes in the morning when patients book same-day appointments, and occasional surges during seasonal illness outbreaks. During these peak

periods, the system must continue processing booking and check-in requests without noticeable delay, even as the number of simultaneous requests rises well above a normal day's traffic.

Separately, clinics operate in different regions and occasionally experience network interruptions. The system must remain usable at a branch even when connectivity to the central system is temporarily unavailable.

The company plans to expand to new clinics regularly, and the system is expected to remain in operation for many years, with periodic upgrades to accommodate new regulations, new clinic workflows, and growing numbers of patients and staff.

1. Identify Quality Attributes

Identify at least four quality attributes that are architecturally significant for this system. For each one, briefly justify why it matters based on the case study description (1–2 sentences per attribute).

Example (do not reuse this exact attribute):

- Availability — clinics must stay usable during network interruptions with the central system

2. Quality Attribute Scenarios

Choose two of the quality attributes you identified above. For each, write a quality attribute scenario using the structure below.

Element	Description
Source	Who or what originates the event - a user or another system
Stimulus	The action or external event that arrives at the system
Environment	The external circumstances under which the requirement must be met
Artifact	The part of the system the requirement applies to
Response	How the system reacts to the stimulus
Response Measure	A metric that quantifies and tests the response

Example (for format only — do not reuse this exact scenario):

Availability scenario

Element	Description (Availability)
Source	External to the system — loss of network connectivity between a branch clinic and the central system
Stimulus	Fault: connection to the central system is lost (omission fault)
Environment	Normal operating hours at a branch clinic
Artifact	Appointment booking and patient check-in module

Response	System detects the loss of connectivity, notifies staff that it is running in degraded (offline) mode, and continues accepting bookings and check-ins locally
Response Measure	• System must be available during clinic operating hours (7:00 AM – 8:00 PM, 7 days a week) • Can operate in degraded (offline) mode for up to 4 hours during an outage • All locally captured records sync with the central system within 10 minutes of reconnection (repair time)

Some Guidelines

- Do not mention specific technologies, frameworks, or programming languages.
- **Focus on measurable, testable qualities** rather than vague terms like "should be fast" or "should be secure."
- Not every requirement is architecturally significant — select only the ones that genuinely shape the architecture.
- Keep answers clear, concise, and well-reasoned.
- Diagrams are not required for this activity.
- Do the thinking on your own and use AI/LLM only to organize your thoughts. Do not blindly depend on LLMs.